

Ejercicio 01: dataset público y SVM reproducible

INF-8239 Ciencia de Datos II, Unidad 01 (U01.E01)

Estudiante	Roque Alexis Ventura Vallejo
Matrícula	100805873
Repositorio	github.com/RoqueVentura/UASD-Maestria-Ciencia-De-Datos , carpeta Ciencia de datos II/Unidad I

1. Dataset

Trabajé con HCV Data, un conjunto de analíticas de sangre de donantes y de pacientes con hepatitis C publicado en el repositorio UCI por Lichtinghagen, Klawonn y Hoffmann (DOI 10.24432/C5D612). Tiene licencia CC BY 4.0, que permite el uso académico citando la fuente, y fue aprobado por el docente. Se descarga directamente desde UCI con la función `download_csv` del paquete del proyecto.

La pregunta es si una SVM puede identificar enfermedad hepática a partir de una analítica de rutina, para decidir qué casos pasan a revisión. Cada fila es una persona con su analítica, edad y sexo. El target hepatopatía vale *si* para hepatitis, fibrosis o cirrosis (75 casos) y *no* para los donantes sanos (533). El error más costoso es el falso negativo, es decir, un enfermo clasificado como sano.

También evalué Maternal Health Risk. La tabla resume por qué lo descarté:

Criterio	HCV Data (seleccionado)	Maternal Health Risk
Filas y columnas	615 y 14	1014 y 7, con solo 452 filas únicas
Target	5 categorías, agrupadas en 75 enfermos y 533 sanos	3 niveles de riesgo (406, 336 y 272)
Ausentes	31 celdas, sobre todo en ALP y CHOL	Ninguno
Problemas	Identificador que filtra el target (corregido)	55 % de filas duplicadas y un pulso registrado de 7 lpm

2. Auditoría y fuga de información

No hay filas duplicadas. Los 31 valores ausentes se concentran en pacientes enfermos, por lo que los imputé con la mediana dentro del pipeline y no antes de la partición. El hallazgo más importante fue que la columna `Unnamed: 0` está ordenada por diagnóstico: los donantes van de la fila 1 a la 533 y los enfermos de la 541 a la 615. Un modelo que solo usa esa columna obtiene $F1 = 1.0$, así que la eliminé. También quité `Category`, que es la fuente del target, y las 7 filas de donantes sospechosos, cuya etiqueta no es fiable. El significado, la unidad y el riesgo de cada variable están en `docs/diccionario_datos.md`.

3. Análisis de dimensionalidad

El archivo original tiene 615 filas y 14 columnas. La tabla clasifica cada variable según su tipo y explica si entra en el modelo:

Variable	Descripción	Tipo	Uso	Motivo o tratamiento
Unnamed: 0	Número de fila	Otro (identificador)	Descartada	Está ordenada por diagnóstico y por sí sola predice el target ($F1 = 1.0$)
Category	Diagnóstico en 5 categorías	Cualitativa ordinal	Descartada	Es la fuente del target; usarla como predictor sería dar la respuesta al modelo
Age	Edad en años	Cuantitativa discreta	Usada	Estandarización
Sex	Sexo (m/f)	Cualitativa nominal binaria	Usada	One-hot: genera 2 columnas

Variable	Descripción	Tipo	Uso	Motivo o tratamiento
ALB, ALP, ALT, AST, BIL, CHE, CHOL, CREA, GGT, PROT	Pruebas bioquímicas en sangre (proteínas, enzimas hepáticas, bilirrubina, colesterol, creatinina)	Cuantitativas continuas (10)	Usadas	Imputación por mediana (ALB, ALP, ALT, CHOL y PROT tienen ausentes) y estandarización
hepatopatía	Target derivado de Category	Cualitativa nominal binaria	Variable objetivo	si = hepatitis, fibrosis o cirrosis; no = donante sano

Quedan 12 predictores: 11 cuantitativos y 1 cualitativo. Después del one-hot de Sex, el modelo recibe 13 columnas. Además de las dos columnas, se descartan 7 filas de donantes sospechosos porque su etiqueta es incierta, lo que deja 608 filas y 486 para entrenamiento. Eso da unas 37 observaciones por columna, una dimensión baja para una SVM.

Revisé si convenía reducir la dimensión. Las correlaciones entre las variables numéricas son moderadas: la más alta es 0.51, entre AST y GGT, seguida de ALP y GGT (0.45) y de ALB y PROT (0.43). En un PCA ajustado solo con el conjunto de entrenamiento, el primer componente explica el 21.7 % de la varianza y hacen falta 9 de los 11 componentes para llegar al 90 %. La información está repartida entre casi todas las variables, así que no apliqué reducción de dimensionalidad: se perdería información a cambio de muy poca simplificación.

4. Resultados

Usé una partición estratificada 80/20 con random_state=42. La línea base y la SVM comparten la partición y el preprocesamiento (imputación, estandarización y one-hot en un ColumnTransformer).

Modelo (conjunto de prueba)	F1-macro	Accuracy	Recall enfermos	FN	FP
HCV: línea base (clase más frecuente)	0.467	0.877	0.00	15	0
HCV: SVM RBF (C=1, gamma=scale)	0.844	0.943	0.60	6	1
LAB01 Breast Cancer: línea base	0.387				
LAB01 Breast Cancer: SVM ajustada (C=10, gamma=0.01)	0.981	0.982		1	1

En HCV, FN es un enfermo clasificado como sano. En LAB01 la SVM ajustada obtuvo 0.974 de F1-macro en validación cruzada.

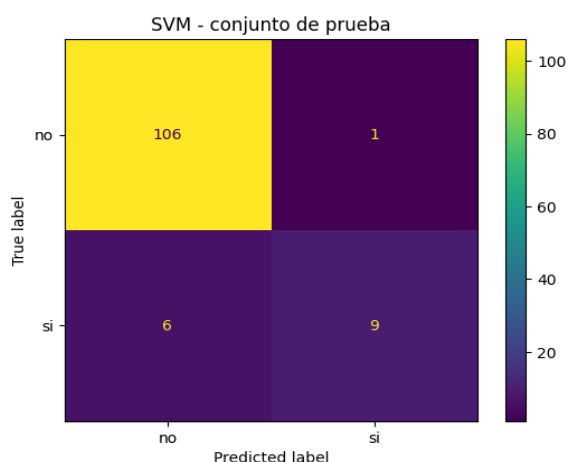


Figura 1. HCV, SVM en prueba: detecta 9 de 15 enfermos.

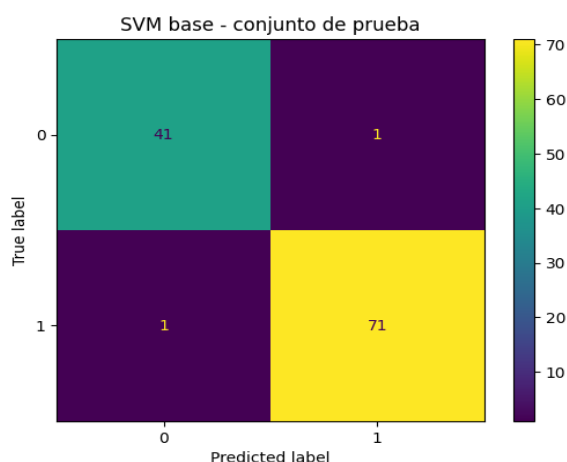


Figura 2. LAB01, SVM base: 2 errores en 114 casos.

5. Código y repositorio

El código reutilizable está en `src/inf8239_u01`: `build_svm()` construye el pipeline y `download_csv()` descarga y valida el dataset. El README explica la instalación del entorno virtual, la descarga y la ejecución. Trabajé en Windows 11 con Python 3.10.11 y scikit-learn 1.7.2, y el historial de Git tiene un commit por cada etapa.

6. Pruebas automatizadas

Las pruebas usan pytest y se ejecutan desde la raíz del proyecto con `python -m pytest -q`, con `PYTHONPATH=src`. Las del contrato de datos necesitan que el CSV ya esté descargado.

Archivo	Prueba	Qué comprueba
test_environment.py	test_environment_message	El paquete se importa y el entorno responde
test_models.py	test_svm_returns_one_prediction_per_row	La SVM entrena y devuelve una predicción por fila
test_models.py	test_svm_rejects_non_positive_c	<code>build_svm</code> rechaza $C = 0$ con <code>ValueError</code>
test_models.py	test_pipeline_contains_scaler_and_model	El pipeline escala antes del modelo (sin fuga)
test_data_contract.py	test_dataset_is_not_empty	El dataset descargado tiene filas
test_data_contract.py	test_required_columns_exist	Existen Category, Age, Sex, ALT, AST y GGT
test_data_contract.py	test_target_has_no_missing_and_two_classes	El target no tiene ausentes y tiene al menos dos clases

Resultado: las 7 pruebas pasan (7 passed en 1.4 s, pytest 9.1.1).

7. Conclusión

Elegí el dataset HCV Data de UCI (CC BY 4.0) porque permite plantear un problema de tamizaje realista: decidir, a partir de una analítica de sangre de rutina, qué personas deberían pasar a revisión hepatológica. Lo comparé con Maternal Health Risk, que descarté porque más de la mitad de sus filas están duplicadas y solo le quedan 452 observaciones únicas.

La auditoría fue lo que más aportó. La columna `Unnamed: 0`, que parece un simple índice, está ordenada por diagnóstico: los donantes ocupan las primeras 533 filas y los enfermos las últimas. Un árbol entrenado solo con esa columna obtiene un F1 de 1.0, así que la eliminé antes de modelar. También retiré `Category`, de la que sale el target, y las siete filas de donantes sospechosos, porque su etiqueta no es fiable. Los valores ausentes son pocos, pero aparecen sobre todo en pacientes enfermos; los imputé con la mediana dentro del pipeline para que solo se aprendan del conjunto de entrenamiento.

Con la misma partición estratificada y el mismo preprocesamiento, la línea base obtuvo un F1-macro de 0.467 y la SVM sin ajustar llegó a 0.844. La exactitud de la SVM es de 0.94, pero esa cifra engaña: de los 15 enfermos del conjunto de prueba, el modelo solo detectó 9. En este problema ese es el error que más importa, porque un paciente enfermo clasificado como sano se queda sin seguimiento. En cambio, entre los 107 donantes hubo una sola falsa alarma.

La causa principal es el desbalance. Solo el 12 % de las observaciones son positivas, y una SVM sin ajustes tiende a favorecer a la clase mayoritaria. El siguiente paso es probar `class_weight="balanced"` y buscar `C` y `gamma` con validación cruzada estratificada, reservando el conjunto de prueba para la evaluación final.

Hay que leer estos resultados con cautela: con 15 positivos en prueba, cada acierto mueve el recall casi siete puntos, los datos provienen de un solo centro y UCI no publica las unidades de las variables. El trabajo es académico y no sustituye un diagnóstico médico.

8. Uso de inteligencia artificial

Herramienta	Claude Code (modelo Claude Opus 5.5, de Anthropic), usado desde Visual Studio Code en septiembre de 2026.
Tareas asistidas	Revisar que el proyecto cumpliera las guías LAB00 a LAB02, proponer y comparar los datasets candidatos, generar el notebook 02, las pruebas del contrato de datos y la documentación (ficha, diccionario, README), y redactar el borrador de la conclusión y de este informe.
Decisiones propias	Elegí el dominio de salud, acepté el dataset propuesto. Ejecuté los notebooks, hice los commits y publiqué el repositorio. Revisé los textos y pedí cambios, como acortar la conclusión y añadir el análisis de dimensionalidad.
Verificación	Las cifras del informe provienen de ejecutar los notebooks y las pruebas en mi entorno. Comprobé que las 7 pruebas pasan y que los resultados coinciden con los notebooks. Las citas se revisaron en las fuentes originales.
Responsabilidad	Revisé el contenido final y asumo la responsabilidad de lo entregado. Las unidades de las variables son inferidas y están señaladas para verificar.

9. Fuentes

- Hoffmann, G., Bietenbeck, A., Lichtinghagen, R., & Klawonn, F. (2018). Using machine learning techniques to generate laboratory diagnostic pathways: A case study. *Journal of Laboratory and Precision Medicine*, 3. <https://doi.org/10.21037/jlpm.2018.06.01>
- Lichtinghagen, R., Klawonn, F., & Hoffmann, G. (2020). *HCV data* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5D612>
- Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1995). *Breast Cancer Wisconsin (Diagnostic)* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5DW2B>